

TrailWise AI — Project Design Document

SE3090 — Software Engineering Frameworks

Assignment 1: Integrated Full-Stack and Agentic AI Application

TrailWise — Tour Management & Travel Planning System

(Domain: Tourism Operations — Booking, Guides & Fleet Coordination)

1. Business Problem & Overview

Local tour operators and travel agencies typically coordinate tour bookings, guide assignments, vehicle and transport allocation, and itinerary changes manually — through phone calls, spreadsheets, and disconnected messaging. A traveler requesting a custom trip has to wait for a staff member to manually check which guides speak the right language and are free on the right dates, which vehicles can fit the group, and whether the whole plan still fits the traveler's budget. This causes double-booked guides, vehicle conflicts, last-minute itinerary problems, and poor communication once a trip is underway.

TrailWise is a Smart Tour Management & Travel Planning System that picks up the moment a traveler requests a tour, and owns everything after that: matching a suitable guide, checking vehicle capacity, pricing the trip correctly, checking the weather and route, and — when a request falls outside standard policy — escalating to a real business decision that a human must approve. Traveler-facing browsing and initial requests stay simple and self-service; TrailWise's Agentic AI subsystem owns matching, validation, and proposal-building from "Requested" onward.

Each tour package is offered in multiple tiers — First, Second, or Normal class, with or without food included — and each vehicle has its own attributes (air conditioning, seat configuration). This means the system isn't just matching "a guide" and "a vehicle" to a booking; it has to resolve tier-specific eligibility and pricing rules, which is what gives the Agentic AI workflow a genuine multi-constraint matching problem to solve rather than a single lookup.

1.1 Why This Domain Fits the Assignment

- Four distinct roles with genuinely different responsibilities and permissions (Traveler, Tour Guide, Operations Manager, Fleet Coordinator).
- Four naturally separable business components — one per student — each with its own entities and business-specific operations.
- Three distinct high-impact actions that must pause for human approval — a large-group/custom itinerary, a budget-override quotation, and a late-cancellation refund exception — genuinely more than the spec's minimum of one, giving a richer, more defensible viva story.
- A real multi-constraint matching problem for the AI to solve: finding a guide who is simultaneously free, language-matched, and specialization-matched, while a vehicle is simultaneously available and large enough — a stronger reasoning task than a single-lookup workflow.
- Rich data for reporting/analytics: occupancy rate, guide utilization, revenue by package, cancellation rate.
- A strong Flutter use case: travelers browsing and booking on the go, and guides checking assigned tours and marking attendance in the field.
- Low-cost/no-cost third-party services available (free-tier maps API, free-tier weather API, payment gateway sandbox), satisfying the "no paid subscription required" rule.

2. User Roles & Permissions

This satisfies the assignment's minimum of at least three roles with different responsibilities and permissions (Section 4.1 of the spec) — the system uses four primary roles, each with a clearly distinct scope of action and a distinct

relationship to the approval workflow. An Administrator role manages users, roles and system settings as a shared, cross-cutting feature and is not counted as one of the four business components.

Role	Description	Key Permissions
Traveler / Customer	Browses tour packages, submits custom tour requests with dates/group size/budget, tracks booking status.	Create booking requests, view own bookings and itinerary, upload documents, submit reviews.
Tour Guide	Owns delivery of the tour itself. Views assigned tours, updates personal availability, logs tour completion and notes.	View assigned bookings, update availability calendar, mark attendance/completion, add tour notes.
Operations Manager	Approves every high-impact action: large-group/custom itineraries, budget-override quotations, and late-cancellation refund exceptions.	Approve/reject/revise AI proposals, view company-wide occupancy and revenue dashboards, manage tour packages.
Fleet Coordinator	Owns vehicle and driver resources. Manages vehicle maintenance status and reviews AI-proposed vehicle assignments.	Manage vehicles and drivers, view/adjust vehicle assignments, flag vehicles as unavailable for maintenance.

3. Component Ownership (4 Students, 4 Primary Components)

Each student owns one full vertical slice — backend endpoints, database entities, a React operations-console view, a Flutter screen set, tests, and one distinct Agentic AI agent contribution — satisfying the one-component-per-student rule.

Student	Primary Component	Core Entities	Sample Business-Specific Operation
Student 1	Tour Package & Booking Management	TourPackage, PackageTier, Booking, BookingAddOn	Validate a booking request against package tier availability (class, with/without food), group capacity, and traveler-stated budget, then initiate the Agentic AI workflow.
Student 2	Guide & Itinerary Management	Guide, GuideAvailability, ItineraryStep	Match a guide's specialization and language to the requested tour theme, and detect availability conflicts.
Student 3	Fleet & Transport Management	Vehicle, VehicleAssignment, Driver	Reserve a vehicle transactionally for the requested dates, matching AC and seat-configuration requirements to the booking's package tier, and prevent double-booking via overlap checks.
Student 4	Payments, Reviews & Reporting	Payment, Discount, Review, AuditLog	Calculate the full tour cost (base + add-ons + group discounts) and generate an occupancy/revenue report.

Note: every student still touches the full stack (backend, DB, React, Flutter, tests) for their component, plus contributes one of the four distinct Agentic AI agents described in Section 8 — this is required by the spec's individual-evidence table.

4. Database Design (PostgreSQL)

A normalised relational schema with clear foreign-key relationships across the four components. Core entities:

Table	Key Columns	Notes
Users	Id, Name, Email, PasswordHash, Role	Roles: Traveler, TourGuide, OperationsManager, FleetCoordinator, Admin
TourPackages	Id, Name, Theme, DurationDays, BasePricePerPerson, MaxGroupSize	Predefined package templates (cultural, adventure, nature, etc.)
PackageTiers	Id, TourPackageId, ClassType, IncludesFood, BasePricePerPerson, RequiresAC	ClassType: First, Second, Normal; one package has multiple tiers. RequiresAC flags tiers (e.g. First Class) that require an AC vehicle.
Bookings	Id, TravelerId, TourPackageId, PackageTierId, GroupSize, StartDate, EndDate, BudgetPerPerson, Status	Status: Requested, PlanProposed, PendingApproval, Confirmed, Completed, Cancelled. PackageTierId drives pricing and vehicle-matching rules.
BookingAddOns	Id, BookingId, Description, Cost	Optional extras (e.g. private transport upgrade, extra activity)
Guides	Id, Name, Languages, Specializations, ContactInfo	Specializations: Cultural, Adventure, Nature, Culinary, etc.
GuideAvailability	Id, GuideId, Date, IsAvailable, AssignedBookingId	Drives the Guide Matching Agent's conflict-checking
Vehicles	Id, Type, Capacity, HasAC, SeatConfiguration, MaintenanceStatus	Type: Van, Coach, SUV. SeatConfiguration e.g. '2+2', '2+3', Luxury/Standard — used to match against tier requirements.
Drivers	Id, Name, LicenseNumber, ContactInfo	Linked to VehicleAssignments
VehicleAssignments	Id, VehicleId, BookingId, DriverId, StartDate, EndDate	Prevents double-booking via date-range overlap check
Itineraries	Id, BookingId, DayNumber, Activity, Location, StartTime	Day-by-day plan generated for the confirmed booking
Payments	Id, BookingId, Amount, Method, PaidAt, Status	Status: Pending, DepositPaid, FullyPaid, Refunded
Reviews	Id, BookingId, Rating, Comment, SubmittedAt	Post-tour traveler feedback
AgentWorkflowRuns	Id, BookingId, Objective, PlanJson, Status, StartedAt, CompletedAt	Persisted Agentic AI workflow state (Section 9 requirement)
AgentStepLogs	Id, WorkflowRunId, AgentName, InputJson, OutputJson, ToolCallsJson, ValidationResult, DurationMs	Auditable execution summary / observability
AuditLogs	Id, EntityType, EntityId, Action, PerformedBy, Timestamp, Details	Full system audit trail for the Operations Manager

- All tables include CreatedAt / UpdatedAt audit fields.
- Foreign keys: Bookings→TourPackages/Users, BookingAddOns→Bookings, VehicleAssignments→Vehicles/Bookings/Drivers, Itineraries→Bookings, Payments→Bookings, AgentStepLogs→AgentWorkflowRuns.
- Transactions used when: confirming a booking must succeed or fail together with assigning the guide, reserving the vehicle, and writing an AuditLog entry.

- Indexes on Bookings.Status, Bookings.TravelerId, GuideAvailability.GuidId+Date, VehicleAssignments.VehicleId+StartDate for fast dashboard filtering and conflict checks.

5. ASP.NET Core Backend — Endpoint Groups

Area	Example Endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
Tour Packages & Bookings	GET/POST /api/packages, GET/POST /api/bookings, GET /api/bookings/{id}, PATCH /api/bookings/{id}/status
Guides & Itinerary	GET/POST /api/guides, GET /api/guides/{id}/availability, POST /api/bookings/{id}/itinerary, GET /api/bookings/{id}/itinerary
Fleet & Transport	GET/POST /api/vehicles, POST /api/bookings/{id}/assign-vehicle, GET /api/vehicles/{id}/availability
Approvals	GET /api/approvals/pending, POST /api/approvals/{id}/decide
Agentic AI	POST /api/agent-workflows/start, GET /api/agent-workflows/{id}, POST /api/agent-workflows/{id}/approve, GET /api/agent-workflows/{id}/summary
Payments & Reviews	POST /api/payments, GET /api/bookings/{id}/payment-status, POST /api/bookings/{id}/reviews
Reporting	GET /api/reports/occupancy, GET /api/reports/revenue, GET /api/reports/guide-utilization

- JWT authentication with role claims; [Authorize(Roles="OperationsManager")] -style protection on approval and reporting endpoints.
- Global exception-handling middleware returning consistent problem-details responses.
- Server-side validation on all write DTOs (group size > 0, dates in range, valid package/vehicle references).
- Swagger/OpenAPI enabled for all controllers; CORS configured for the deployed React origin.

6. React Web Application (Operations Console)

Primary users: Operations Manager, Fleet Coordinator, Tour Guide (status views).

- Dashboard — upcoming tours, pending-approval count, guide and vehicle utilization at a glance.
- Package & booking management — searchable/filterable/paginated booking list, package catalogue with tier configuration (class type, food inclusion, tier pricing).
- Guide and vehicle availability calendars, including vehicle AC/seat-configuration attributes for tier-matching.
- Approval queue — three distinct approval types (large-group/custom itinerary, budget override, refund exception) wired to the Agentic AI approval endpoint.
- Agent workflow monitor — live status of each AgentWorkflowRun (plan, steps completed, tool calls, validation results) with an execution-summary drill-down.
- Itinerary builder/editor for confirmed bookings.
- Reports — occupancy chart, revenue by package, guide utilization, exportable audit report.
- State management: React Context + hooks for auth/session, and either Redux Toolkit or Zustand for booking/approval state — decision to be justified in the ADR.

7. Flutter Mobile Application (Traveler & Guide App)

Primary users: Travelers browsing and booking tours, with a guide-side view for assigned tours.

- Login / registration with secure token storage (flutter_secure_storage).

- Tour browsing and custom request form — dates, group size, budget, theme, language preference, and package tier selection (class type, with/without food).
- Document upload via camera or file picker (ID/passport for booking).
- GPS/maps for meeting-point directions and live tour location.
- Booking status tracking, itinerary viewer, and push notifications.
- Guide-side view: assigned tours, attendance/completion marking, tour notes.
- Responsive states: loading, empty ('no tours booked'), and error (offline) handling.
- State management: Provider or Riverpod — justified in the ADR alongside the React choice for consistency of reasoning.

Device features used: camera/file picker (document upload), GPS/maps (meeting points and live location), and date/time selection (tour date picking) — satisfying the spec's requirement for at least one meaningful device feature.

8. Agentic AI Subsystem — 4 Distinct Agents

The assessed workflow begins the moment a traveler submits a tour request, and ends when the booking is fully confirmed — or, for exceptions, when a human has made and recorded a real business decision.

8.1 Agent Roster

Agent	Responsibility	Inputs	Outputs / Tools
Coordinator / Planning Agent	Receives the traveler's objective, builds a structured multi-step plan, and delegates each step to the correct specialist agent.	Booking-request event (dates, group size, theme, budget, language)	Structured plan (JSON) persisted to AgentWorkflowRuns.PlanJson; delegation calls to the other three agents
Guide Matching Agent	Finds a guide who is simultaneously available, language-matched, and specialization-matched to the tour theme.	Booking details + Guide/GuideAvailability records	Structured JSON {guideld, matchScore, reasoning}; allow-listed DB-read tool on Guides/GuideAvailability only
Fleet & Capacity Agent	Checks vehicle availability and capacity for the group size across the requested dates, filtering candidates by AC and seat-configuration requirements driven by the booking's package tier. Cannot reserve the vehicle until the proposal is approved.	Booking details + PackageTier requirements + Vehicle/VehicleAssignment records	Structured JSON {vehicleId, driverId, acMatch, seatConfigMatch, conflictCheck}; allow-listed DB-read tool, write gated behind approval
Pricing & Validation Agent	Calculates the full tier-aware quotation (tier base price + optional catering cost + add-ons – group discounts) and validates the whole proposal against policy, tier-vehicle compatibility rules, and the traveler's budget.	Booking + PackageTier + guide proposal + vehicle proposal + pricing rules	Structured JSON {totalCost, breakdown, validationResult}; schema-validator tool

8.2 Minimum Assessed Workflow (mapped to Section 9.1 requirements)

- 1 Traveler submits a tour request through Flutter (dates, group size, theme, budget, language preference, and chosen package tier — e.g. First Class, with food); the Coordinator/Planning Agent reads it and creates the Booking record with Status = Requested.
- 2 Coordinator Agent creates a plan: match guide → check tier-compatible vehicle capacity → calculate tier-aware pricing → validate → request approval.
- 3 Guide Matching Agent proposes a guide with reasoning; if no ideal guide is free, it flags the conflict for the coordinator.
- 4 Fleet & Capacity Agent checks vehicle availability and capacity for the group size and dates, filtering candidates whose HasAC and SeatConfiguration satisfy the booking's PackageTier (e.g. a First Class tier requires HasAC = true), and proposes a matching vehicle and driver.
- 5 Pricing & Validation Agent calculates the total quotation — tier base price + catering cost if the tier includes food + add-ons – group discounts — and runs deterministic checks: guide specialization/language match, sufficient vehicle capacity, tier-vehicle compatibility (e.g. First Class tier must be assigned an AC vehicle), no scheduling conflicts, budget compliance, correct cost calculation, and required JSON schema.

- 6 If the proposal is a standard-size booking within budget, it can auto-confirm. If it is a large-group/custom itinerary, exceeds the traveler's budget beyond a policy margin, or is a late-cancellation refund request, the workflow pauses — status becomes PendingApproval — and routes to the Operations Manager.
- 7 Operations Manager reviews the evidence (guide match reasoning, vehicle assignment, full pricing breakdown, validation results) in React and approves, rejects, or requests revision.
- 8 On approval, the workflow resumes, writes an auditable execution summary (steps, tool calls, timings, validation results, decision) to AgentWorkflowRuns / AgentStepLogs, assigns the guide, reserves the vehicle in a database transaction, and the booking status updates to Confirmed.
- 9 The Flutter app (Traveler) receives the confirmed itinerary, guide details, and payment status — completing the required end-to-end cross-platform loop (Flutter request → API → PostgreSQL → Agentic AI → React approval → Flutter status update).

8.3 The Three Approval Types (why this domain has more than the minimum)

Approval Type	Triggered By	Approver
Large-Group / Custom Itinerary	Group size exceeds the auto-approval threshold, or the traveler requests an itinerary that deviates from a standard package.	Operations Manager
Budget-Override Quotation	The calculated total cost exceeds the traveler's stated budget by more than the policy-defined margin.	Operations Manager
Cancellation / Refund Exception	A traveler cancels close to the start date and requests a refund outside the standard cancellation policy.	Operations Manager

8.4 Controls, Validation & Security

- Allow-listed tools only: guide-availability read tool, vehicle-availability read/write tool (write gated behind approval), pricing calculator, structured-output formatter. No unrestricted web or file-system access.
- Every tool input is schema-validated before use; every tool output is schema-validated before being trusted.
- Deterministic business-rule checks (capacity limits, budget-override thresholds, cancellation-window timing, and tier-vehicle compatibility — e.g. a First Class booking must be assigned a vehicle with HasAC = true) run in code — not left to the LLM's judgement — before any high-impact action is allowed.
- Human approval is mandatory for: any large-group/custom itinerary, any budget-override quotation, and any refund exception.
- Timeouts and retry limits on the maps and weather API calls; safe-failure path (booking marked NeedsManualReview) if a proposal cannot be confidently validated.
- Prompt-injection resistance: free-text special requests from travelers are treated as untrusted data, never as instructions — the agents' system prompts explicitly ignore any instruction-like text found inside a traveler's request.
- No secrets, tokens, or full payment card details are persisted in AgentWorkflowRuns or AgentStepLogs — only the structured fields needed for audit.

9. Third-Party Integration

Option	Purpose	Notes
Maps API (e.g. Google Maps or OpenRouteService)	Route planning, meeting-point directions, and live location sharing during the tour.	Primary recommendation — free tier is sufficient for evaluation.
Weather API (e.g. OpenWeatherMap)	Checks the forecast for outdoor-heavy tours and feeds risk information into the itinerary proposal.	Free tier; graceful fallback (skip weather check) if the service is unavailable.
Payment Gateway Sandbox (e.g. Stripe test mode)	Simulates deposit and full-payment processing for confirmed bookings.	Free sandbox mode; no real money moves, satisfying the "no paid subscription" rule.

- All third-party calls are routed through the ASP.NET Core backend — never called directly from React or Flutter.
- API keys/credentials stored in environment variables / secret configuration, never committed to Git.
- Timeouts, retry limits, and a graceful fallback (manual entry) if the maps, weather, or payment service is unavailable.

10. Technology Stack Summary

Layer	Technology
Backend	C# / ASP.NET Core Web API, Entity Framework Core
Database	PostgreSQL
Web App	React (functional components, hooks, React Router, Redux Toolkit or Zustand)
Mobile App	Flutter / Dart (Provider or Riverpod), notifications, date/time picker, GPS
Agentic AI	LangGraph (or Microsoft Agent Framework / custom orchestration) + a locally-run or free-tier LLM
Auth	JWT + role-based authorization
CI/CD	GitHub Actions (build + backend test workflow on every PR to main)
Testing	xUnit/NUnit (backend), React Testing Library, Flutter test/widget tests, k6 or similar for performance
Deployment	Free-tier cloud host for API + PostgreSQL, static hosting for React, APK build for Flutter

11. Testing Plan Highlights

- Backend: unit tests per service (e.g. budget-override threshold logic), controller/API integration tests, auth/authorization tests.
- Database: EF Core migration tests, constraint tests (e.g. cannot confirm a booking with an unresolved vehicle conflict), transaction rollback tests.
- React: component tests for the approval queue (all three approval types), protected-route tests, API-integration tests with mocked backend.
- Flutter: widget tests for the booking request form and itinerary viewer, navigation tests.
- End-to-end: the full workflow in Section 8.2, from request through to Confirmed.

- Agent evaluation: golden-case test bookings (clean auto-confirm, large-group requiring approval, budget-override case, injected fake "instruction" text in a traveler's special request to test prompt-injection resistance, unmatchable request to test safe-failure).
- Performance: concurrent booking requests, API response times under load, Agentic AI workflow latency.

12. Suggested Architecture Decision Record (ADR) Topics

- State-management approach in React (Context API vs Redux Toolkit vs Zustand).
- State-management approach in Flutter (Provider vs Riverpod vs Bloc).
- Agentic AI framework and orchestration method (LangGraph vs Microsoft Agent Framework vs custom).
- Database schema strategy for persisting Agentic AI workflow state (dedicated AgentWorkflowRuns/AgentStepLogs tables vs a single JSONB blob).
- Cloud deployment platform for the API, database, and React app.
- Maps/weather API selection and fallback strategy if the free-tier service is unavailable during evaluation.

13. Suggested 9-Week Timeline

Week(s)	Focus
1	Domain confirmation, ER diagram, repo setup, ADR drafts, GitHub Actions CI skeleton.
2–3	Backend core: auth, entities, migrations, base CRUD endpoints per component.
4	React shell + Flutter shell; wire up auth and role-based navigation on both.
5	Component-specific business logic + UI on both clients; start Agentic AI agent scaffolding.
6	Build the 4-agent workflow end-to-end (match guide → check fleet → price → validate); persist state.
7	All three human-approval loops wired into React; cross-platform end-to-end workflow working.
8	Testing pass (unit, integration, E2E, agent evaluation, performance); deployment.
9	Documentation, ADR finalisation, AI usage logs/reflections, demo rehearsal, submission.

14. Team Task Division: Component & Agentic AI Pairing

Each team member owns one full vertical slice of the system, and — deliberately — their Agentic AI agent is built on top of the exact same entities they already own. This means no one has to learn someone else's data model to build their agent: the agent is simply the "smart" version of the component the person already knows inside out.

14.1 Component-to-Agent Mapping

Person	Owns These Entities	Their Agentic AI Agent	What the Agent Does
Person 1	TourPackage, PackageTier, Booking, BookingAddOn	Coordinator / Planning Agent	Reads the incoming booking request, creates the Booking record, builds the step-by-step plan, and delegates to the other three agents.
Person 2	Guide, GuideAvailability, ItineraryStep	Guide Matching Agent	Searches their own Guide/GuideAvailability tables to find a guide who is free, language-matched, and specialization-matched.
Person 3	Vehicle, VehicleAssignment, Driver	Fleet & Capacity Agent	Searches their own Vehicle/VehicleAssignment tables to find a vehicle and driver that satisfy capacity, AC, and seat-configuration requirements.
Person 4	Payment, Discount, Review, AuditLog	Pricing & Validation Agent	Uses their own pricing/discount logic to calculate the total quotation and runs the final validation checks.

14.2 Full Task Breakdown Per Person

Person 1 — Tour Package & Booking Management

- **Entities:** TourPackage, PackageTier, Booking, BookingAddOn
- **Backend:** Package/booking CRUD endpoints; booking validation logic (group size, tier availability, budget check).
- **React:** Package catalogue + tier configuration screen; booking list (searchable/filterable).
- **Flutter:** Tour browsing + custom request form.
- **AI Agent:** Coordinator/Planning Agent — kicks off the whole workflow and delegates to the other three agents.

Person 2 — Guide & Itinerary Management

- **Entities:** Guide, GuideAvailability, ItineraryStep
- **Backend:** Guide endpoints; availability calendar logic; conflict detection.
- **React:** Guide availability calendar view.
- **Flutter:** Guide-side "my assigned tours" + attendance-marking screen.
- **AI Agent:** Guide Matching Agent — finds a guide by language, specialization, and availability.

Person 3 — Fleet & Transport Management

- **Entities:** Vehicle, VehicleAssignment, Driver
- **Backend:** Vehicle endpoints; reservation logic with overlap/double-booking checks.
- **React:** Vehicle availability calendar; AC/seat-configuration attributes view.
- **Flutter:** Supports the meeting-point/GPS device feature (paired with Person 1's traveler screens).
- **AI Agent:** Fleet & Capacity Agent — matches a vehicle to the booking's tier requirements.

Person 4 — Payments, Reviews & Reporting

- **Entities:** Payment, Discount, Review, AuditLog
- **Backend:** Cost calculation logic; payment endpoints; reporting endpoints.
- **React:** Reports dashboard (occupancy, revenue, guide utilization).
- **Flutter:** Payment status screen; review submission.
- **AI Agent:** Pricing & Validation Agent — calculates total cost and runs all validation checks.

14.3 Shared / Cross-Cutting Work (not owned by one person)

- Admin role & auth (JWT setup) — typically owned by whoever sets up the backend skeleton first.
- Approval queue UI in React — all four members should contribute, since it touches every component.
- Agent workflow monitor screen — good fit for whoever is most comfortable with the AI orchestration piece.
- CI/CD (GitHub Actions) and overall ADRs — a joint effort in Week 1 and Week 9.

14.4 Agent Interface Contracts (agree on these early)

Because the Coordinator Agent (Person 1) calls into the other three agents, the team must agree on the exact input/output JSON shape each agent produces — ideally by the end of Week 3–4, before everyone builds their agent in isolation using mock inputs.

Agent	Expected Output Shape
Guide Matching Agent (Person 2)	{ guidId, matchScore, reasoning }
Fleet & Capacity Agent (Person 3)	{ vehicleId, driverId, acMatch, seatConfigMatch, conflictCheck }
Pricing & Validation Agent (Person 4)	{ totalCost, breakdown, validationResult }

14.5 Suggested Order of Work (mapped to the 9-week timeline)

- 1 Week 1: All four members agree on the ER diagram and ADRs together, then split off into their own components.
- 2 Weeks 2–3: Everyone builds their own entities/CRUD endpoints in parallel.
- 3 Week 4: Sync up — wire shared auth/navigation together across React and Flutter.
- 4 Week 5: Each person builds their own agent in isolation, using mocked outputs for the other three agents.
- 5 Week 6 (critical integration week): All four agents are wired together into one live workflow — plan a joint working session.
- 6 Week 7: Approval loops — since these cut across all components, this is another joint week.
- 7 Weeks 8–9: Testing and documentation, split by component, but the end-to-end test requires all four members together.

Key recommendation: decide early who owns the Coordinator Agent, since it depends on the other three agents' output contracts. That person should confirm the JSON shapes above with the rest of the team by the end of Week 4, so no one is blocked during the Week 6 integration push.

This document is a starting scope for group discussion — final entity names, thresholds, and screen lists should be refined together and locked into the ADR before implementation begins.